

CATAM Additional Projects

2.10 Phase and Group Velocity

1 Initial-value problem

Question 1

Let $\hat{x} := x/c_0$, $v(\hat{x}, t) := u(x, t)$ and $\hat{f}(\hat{x}) := f(x)$. Hence $\frac{\partial}{\partial x} = \frac{1}{c_0} \frac{\partial}{\partial \hat{x}}$. Now if $u(x, t)$ satisfies (1) & (2) then we have

$$\begin{aligned} v_{tt}(\hat{x}, t) - c_0^2 v_{xx}(\hat{x}, t) &= -q^2 v(\hat{x}, t) \\ \Rightarrow v_{tt}(\hat{x}, t) - c_0^2 \frac{1}{c_0^2} v_{\hat{x}\hat{x}}(\hat{x}, t) &= -q^2 v(\hat{x}, t) \\ \Rightarrow v_{tt}(\hat{x}, t) - v_{\hat{x}\hat{x}}(\hat{x}, t) &= -q^2 v(\hat{x}, t) \end{aligned}$$

and

$$\begin{aligned} v(\hat{x}, 0) &= \hat{f}(\hat{x}) \\ v_t(\hat{x}, 0) &= 0. \end{aligned}$$

Hence v satisfies (1) with $c_0 = 1$ and (2) with $\hat{f}(\hat{x})$. Thus, working with these new variables, we can without loss of generality set $c_0 = 1$ in equation (1). Now taking the Fourier transform of u with respect to x ,

$$\tilde{u}(k) = \int_{-\infty}^{\infty} e^{-ikx} u(x, t) dx,$$

equation (1) with $c_0 = 1$ becomes:

$$\begin{aligned} \tilde{u}_{tt} - (ik)^2 \tilde{u} &= -q^2 \tilde{u} \\ \Rightarrow \tilde{u}_{tt} &= -(q^2 + k^2) \tilde{u} \\ \Rightarrow \tilde{u} &= A(k) e^{i\Omega(k)t} + B(k) e^{-i\Omega(k)t}. \end{aligned} \tag{1.1}$$

The initial conditions (2) become

$$\begin{aligned} \tilde{u}(k, 0) &= \tilde{f}(k) \\ \tilde{u}_t(k, 0) &= 0, \end{aligned}$$

and combining these with (1.1) we obtain

$$\tilde{u}(k, t) = \frac{1}{2} \tilde{f}(k) [e^{i\Omega(k)t} + e^{-i\Omega(k)t}].$$

Now taking the inverse Fourier transform to find u :

$$\begin{aligned} u(x, t) &= \frac{1}{2\pi} \int_{-\infty}^{\infty} e^{ikx} \tilde{u}(k, t) dk \\ &= \frac{1}{4\pi} \int_{-\infty}^{\infty} \tilde{f}(k) [e^{ikx-i\Omega(k)t} + e^{ikx+i\Omega(k)t}] dk. \end{aligned}$$

By making the substitution $k \rightarrow -k$ and since $\tilde{f}(k)$ and $\Omega(k)$ are both even we find that:

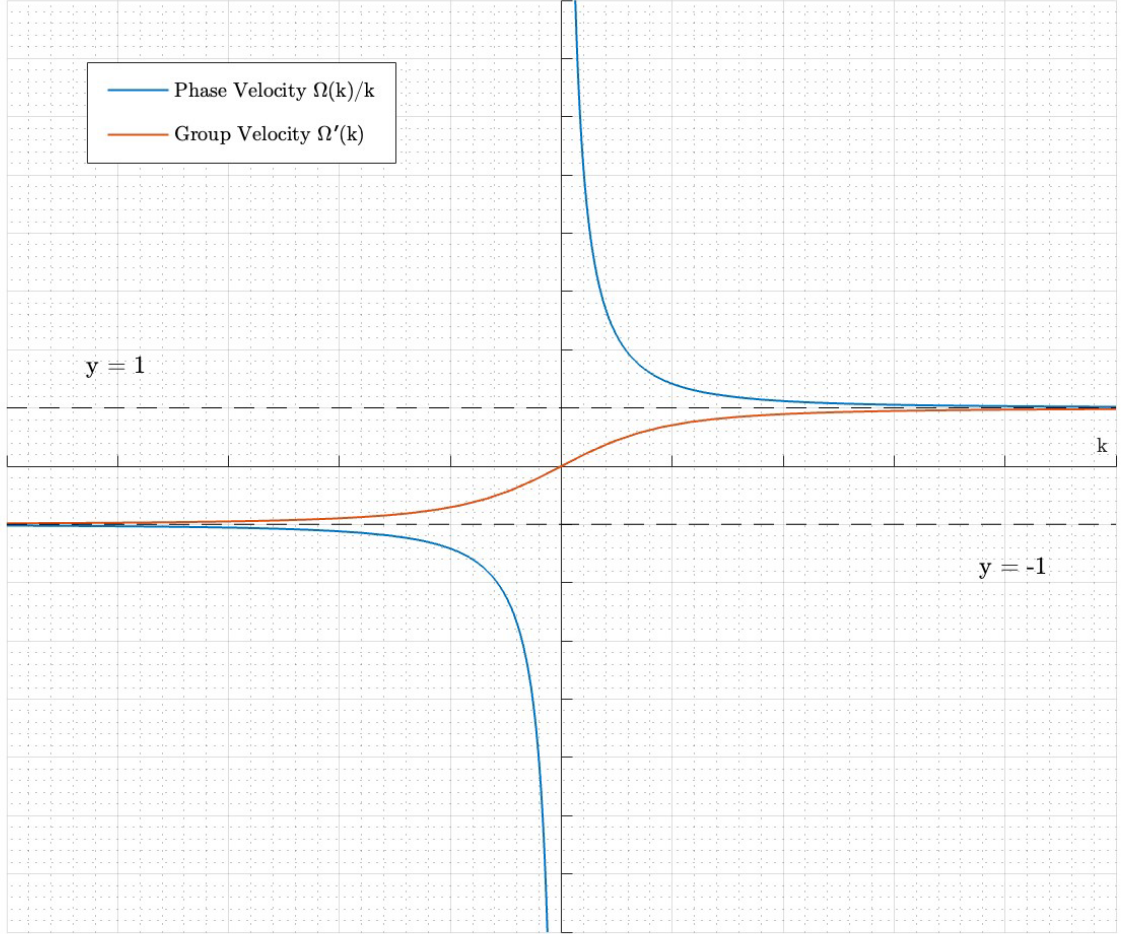
$$\frac{1}{4\pi} \int_{-\infty}^{\infty} \tilde{f}(k) e^{ikx+i\Omega(k)t} dk = \frac{1}{4\pi} \int_{-\infty}^{\infty} \tilde{f}(k) e^{-ikx+i\Omega(k)t} dk.$$

Hence, we have

$$u(x, t) = \frac{1}{4\pi} \int_{-\infty}^{\infty} \tilde{f}(k) e^{ikx - i\Omega(k)t} dk + \text{complex conjugate} .$$

The phase velocity of a wave is the speed at which a point of fixed phase on the wave moves in space. In our 1-dimensional case for example, this would be the speed at which a single crest or trough of $u(x, t)$ moves along the x -axis.

Figure 1.1 *Phase and Group Velocity.*



Question 2

Let

$$u_1(x, t) = \frac{1}{4\pi} \int_{-\infty}^{\infty} \tilde{f}(k) e^{ikx - i\Omega(k)t} dk$$

and hence $u = u_1 + \overline{u_1}$.

Now with $V \equiv x/t$ we write the integrand of u_1 as:

$$u_1 = \frac{1}{4\pi} \int_{-\infty}^{\infty} \tilde{f}(k) e^{it\phi(k)} dk , \quad (2.1)$$

where $\phi(k) = kV - \Omega(k)$ and apply the method of stationary phase as follows:

Noting that

$$\phi'(k) = V - \Omega'(k),$$

we can see from Figure 1.1 that given $|V| < 1$ there will be a unique k_0 satisfying $\Omega'(k_0) = V$ and hence ϕ has a single stationary point at $k = k_0$. The leading order asymptotic behaviour of (2.1) as $t \rightarrow \infty$ is determined by the stationary points of ϕ and in particular we have:

$$u_1 = \frac{1}{4\pi} \int_{k_0-\epsilon}^{k_0+\epsilon} \tilde{f}(k) e^{it\phi(k)} dk + O\left(\frac{1}{t}\right) \quad \text{as } t \rightarrow \infty. \quad (2.2)$$

Furthermore,

$$\phi''(k_0) = -\Omega''(k_0) = -\frac{q^2}{(q^2 + k_0^2)^{\frac{3}{2}}} \neq 0,$$

and so we expect the leading asymptotic to be $O\left(\frac{1}{\sqrt{t}}\right)$.

To obtain this leading order asymptotic for u_1 we approximate f & ϕ in (2.2) about $k = k_0$ by $\tilde{f}(k) \sim \tilde{f}(k_0)$ and $\phi(k) \sim \phi(k_0) + \frac{1}{2}\phi''(k_0)(k - k_0)^2$ to obtain:

$$u_1 \sim \frac{1}{4\pi} \tilde{f}(k_0) e^{it\phi(k_0)} \int_{k_0-\epsilon}^{k_0+\epsilon} e^{\frac{1}{2}it\phi''(k_0)(k-k_0)^2} dk.$$

Making the substitution $s = \sqrt{\frac{-\phi''(k_0)t}{2}} (k - k_0)$, this becomes

$$u_1 \sim \frac{1}{4\pi} \tilde{f}(k_0) e^{it\phi(k_0)} \sqrt{\frac{2}{-\phi''(k_0)t}} \int_{-\epsilon\sqrt{\frac{-\phi''(k_0)t}{2}}}^{+\epsilon\sqrt{\frac{-\phi''(k_0)t}{2}}} e^{-is^2} ds.$$

We proceed by taking the limits of the integral to infinity (which doesn't change the approximation to order $\frac{1}{\sqrt{t}}$) to obtain:

$$u_1 \sim \frac{1}{4\pi} \tilde{f}(k_0) e^{it\phi(k_0)} \sqrt{\frac{2}{-\phi''(k_0)t}} \int_{-\infty}^{+\infty} e^{-is^2} ds \quad \text{as } t \rightarrow \infty.$$

Using standard complex variable methods to evaluate this final integral we obtain the asymptotic approximation

$$u_1 \sim \frac{1}{4\pi} \tilde{f}(k_0) e^{it\phi(k_0)} \sqrt{\frac{2\pi}{-\phi''(k_0)t}} e^{-i\frac{\pi}{4}}.$$

Substituting for ϕ & ϕ'' and simplifying, this becomes

$$\begin{aligned}
u_1 &\sim \frac{1}{2} \tilde{f}(k_0) e^{ik_0 V t - i\Omega(k_0)t} [2\pi\Omega''(k_0)t]^{-\frac{1}{2}} e^{-\frac{i\pi}{4}} \\
&\sim \frac{1}{2} [2\pi\Omega''(k_0)t]^{-\frac{1}{2}} |\tilde{f}(k_0)| \exp \left[i \left(k_0 x - \Omega(k_0)t + \arg \tilde{f}(k_0) - \frac{1}{4}\pi \right) \right].
\end{aligned}$$

Hence,

$$u(x, t) \sim [2\pi\Omega''(k_0)t]^{-\frac{1}{2}} |\tilde{f}(k_0)| \cos \left[k_0 x - \Omega(k_0)t + \arg \tilde{f}(k_0) - \frac{1}{4}\pi \right] \text{ as } t \rightarrow \infty.$$

We now find k_0 in terms of x and t :

$$\begin{aligned}
\Omega'(k_0) &= V \\
\Rightarrow \frac{k_0}{\sqrt{q^2 + k_0^2}} &= \frac{x}{t} \\
\Rightarrow k_0^2 &= \frac{x^2}{t^2} (q^2 + k_0^2) \\
\Rightarrow \left(1 - \frac{x^2}{t^2} \right) k_0^2 &= \frac{q^2 x^2}{t^2} \\
\Rightarrow k_0^2 &= \frac{q^2 x^2}{t^2 - x^2}
\end{aligned} \tag{2.3}$$

By (2.3) k_0 must have the same sign as x and hence we neglect the other square root to obtain:

$$k_0 = \frac{qx}{\sqrt{t^2 - x^2}}.$$

We now substitute this k_0 into (8) in two steps. Firstly:

$$\begin{aligned}
[2\pi\Omega''(k_0)t]^{-\frac{1}{2}} &= \frac{1}{(2\pi)^{\frac{1}{2}}} \frac{1}{t^{\frac{1}{2}}} \left[\frac{q^2}{(q^2 + k_0^2)^{\frac{3}{2}}} \right]^{-\frac{1}{2}} \\
&= \frac{1}{(2\pi)^{\frac{1}{2}}} \frac{1}{t^{\frac{1}{2}}} \frac{\left[q^2 + \frac{q^2 x^2}{t^2 - x^2} \right]^{\frac{3}{4}}}{q} \\
&= \frac{1}{(2\pi)^{\frac{1}{2}}} \frac{1}{t^{\frac{1}{2}}} \frac{1}{q} \left[\frac{q^2 t^2}{t^2 - x^2} \right]^{\frac{3}{4}} \\
&= \frac{q^{\frac{1}{2}} t}{(2\pi)^{\frac{1}{2}} (t^2 - x^2)^{\frac{3}{4}}}.
\end{aligned}$$

Secondly, assuming $\tilde{f}(k)$ is real then

$$\arg \tilde{f}(k_0) = \begin{cases} \pi & \tilde{f}(k_0) < 0 \\ 0 & \tilde{f}(k_0) > 0 \end{cases},$$

and so using the fact that $\cos(\theta + \pi) = -\cos(\theta)$ we have:

$$\cos(\theta + \arg \tilde{f}(k)) = \begin{cases} -\cos(\theta) & \tilde{f}(k_0) < 0 \\ \cos(\theta) & \tilde{f}(k_0) > 0 \end{cases}.$$

Hence,

$$\begin{aligned} & |\tilde{f}(k_0)| \cos \left[k_0 x - \Omega(k_0)t + \arg \tilde{f}(k_0) - \frac{1}{4}\pi \right] \\ &= |\tilde{f}(k_0)| \operatorname{sign}[\tilde{f}(k_0)] \cos \left[k_0 x - \Omega(k_0)t - \frac{1}{4}\pi \right] \\ &= \tilde{f}(k_0) \cos \left[k_0 x - \Omega(k_0)t - \frac{1}{4}\pi \right] \\ &= \tilde{f} \left(\frac{qx}{\sqrt{t^2 - x^2}} \right) \cos \left[\frac{qx^2}{\sqrt{t^2 - x^2}} - \sqrt{q^2 + \frac{q^2 x^2}{t^2 - x^2}} t - \frac{1}{4}\pi \right] \\ &= \tilde{f} \left(\frac{qx}{\sqrt{t^2 - x^2}} \right) \cos \left[\frac{qx^2}{\sqrt{t^2 - x^2}} - \sqrt{\frac{q^2 t^2}{t^2 - x^2}} t - \frac{1}{4}\pi \right] \\ &= \tilde{f} \left(\frac{qx}{\sqrt{t^2 - x^2}} \right) \cos \left[\frac{qx^2 - qt^2}{\sqrt{t^2 - x^2}} - \frac{1}{4}\pi \right] \\ &= \tilde{f} \left(\frac{qx}{\sqrt{t^2 - x^2}} \right) \cos \left[-q\sqrt{t^2 - x^2} - \frac{1}{4}\pi \right] \\ &= \tilde{f} \left(\frac{qx}{\sqrt{t^2 - x^2}} \right) \cos \left[q\sqrt{t^2 - x^2} + \frac{1}{4}\pi \right]. \end{aligned}$$

Putting both steps together we obtain (10):

$$u(x, t) \sim \frac{q^{\frac{1}{2}} t}{(2\pi)^{\frac{1}{2}} (t^2 - x^2)^{\frac{3}{4}}} \tilde{f} \left(\frac{qx}{\sqrt{t^2 - x^2}} \right) \cos \left[q\sqrt{t^2 - x^2} + \frac{1}{4}\pi \right] \quad \text{for } |x| < t.$$

We can view the above asymptotic approximation for u as being composed of two parts; an *amplitude* factor and an *oscillatory* factor. The amplitude factor defines the overall shape (or *envelope*) within which the wave oscillates, whilst the oscillatory factor defines the oscillatory wave motion within the envelope. We interpret the group velocity as the speed at which this envelope moves along the x -axis.

Question 3

A program solving (1) – (2) numerically following the finite-difference scheme outlined in the project manual is listed on page 16.

The centred difference approximation (11) for u_{tt} is accurate to order Δt^2 and hence so is the finite-difference scheme (12). By Taylor expansion of u about $t = 0$ we find:

$$u(x, \Delta t) = u(x, 0) + \Delta t u_t(x, 0) + \frac{1}{2} \Delta t^2 u_{tt}(x, 0) + O(\Delta t^3)$$

$$\begin{aligned}\Rightarrow u_t(x, 0) &= \frac{u(x, \Delta t) - u(x, 0) - \frac{1}{2} \Delta t^2 u_{tt}(x, 0)}{\Delta t} + O(\Delta t^2) \\ &= \frac{u(x, \Delta t) - u(x, -\Delta t)}{2\Delta t} + O(\Delta t^2)\end{aligned}$$

Hence for $u_t(x, 0)$ to equal zero to order Δt^2 we require $u(x, \Delta t) = u(x, -\Delta t)$ which corresponds to $u_i^{-1} = u_i^1$ in our scheme. Similarly, to implement the boundary condition $u_x(0, t) = 0$ to order Δx^2 we set $u_{-1}^j = u_1^j$.

The program solves for u_i^{j+1} using (12) in the form:

$$u_i^{j+1} = -u_i^{j-1} + \left(\frac{\Delta t^2}{\Delta x^2} - \frac{q^2 \Delta t^2}{2} \right) (u_{i-1}^j + u_{i+1}^j) + 2 \left(1 - \frac{\Delta t^2}{\Delta x^2} \right) u_i^j,$$

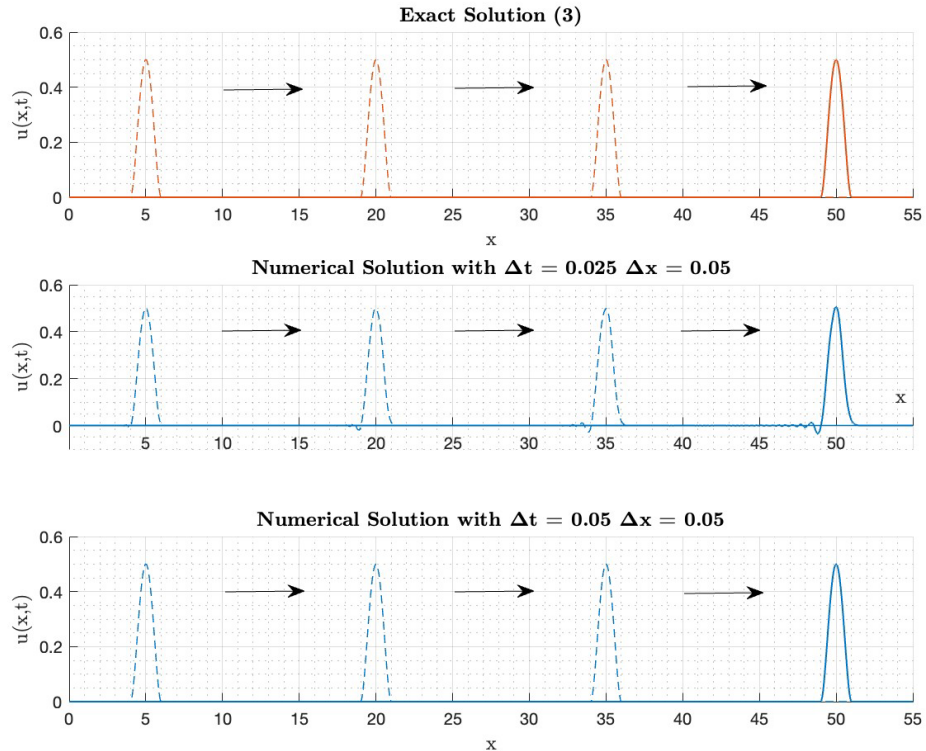
iterating over i from 0 to N and starting from u_i^1 which is found by

$$u_i^1 = \frac{1}{2} \left(\frac{\Delta t^2}{\Delta x^2} - \frac{q^2 \Delta t^2}{2} \right) (u_{i-1}^0 + u_{i+1}^0) + \left(1 - \frac{\Delta t^2}{\Delta x^2} \right) u_i^0$$

using $u_i^0 = f(i\Delta x)$.

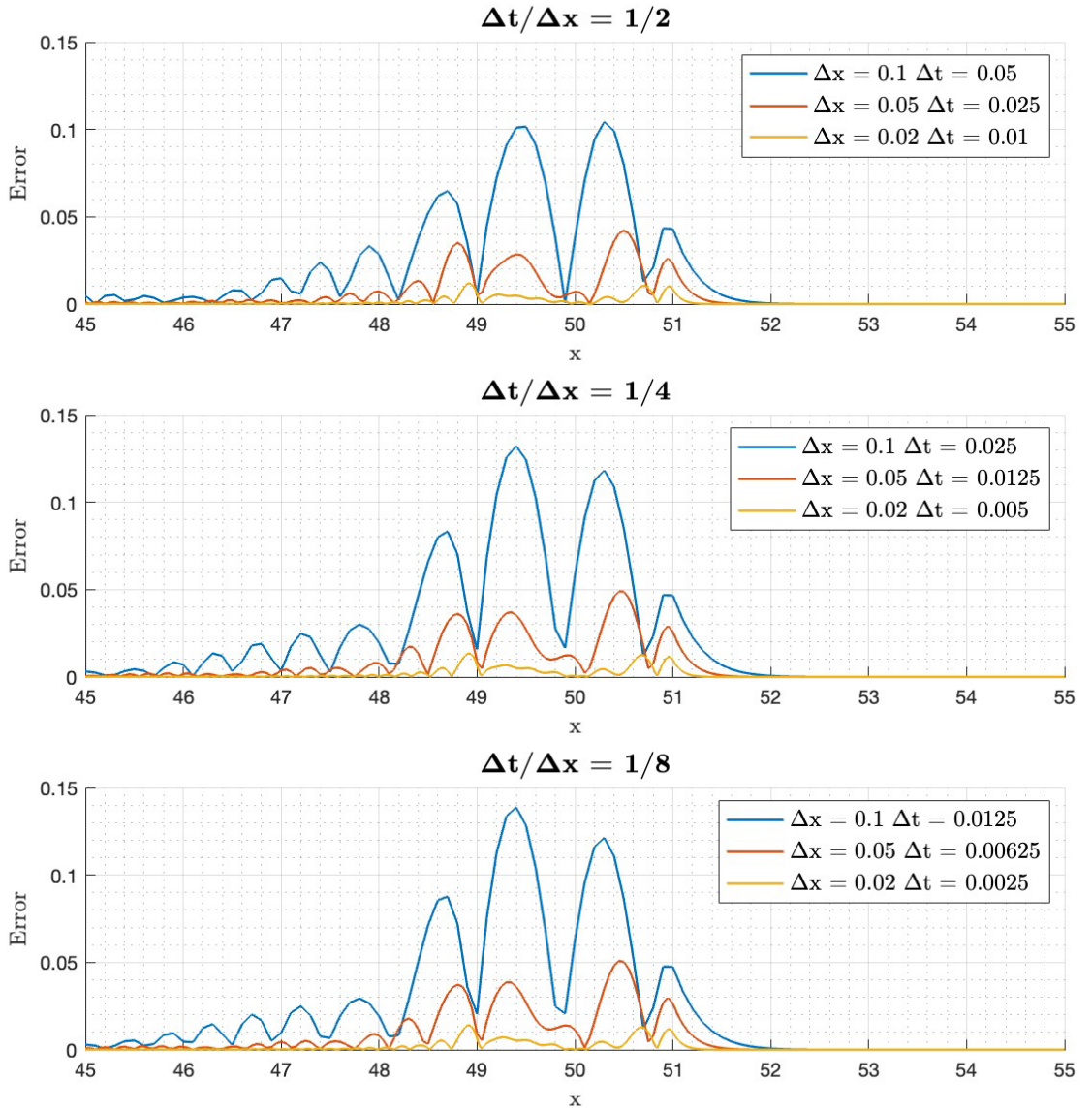
For both $q = 0$ & $q = 1$ we find that the *accuracy* of the numerical scheme depends on the sizes of Δx & Δt and, as will be discussed later, more so on the value of Δx . On the other hand, the *stability* of the numerical method depends on the value of the ratio $\frac{\Delta t}{\Delta x}$. In particular, the numerical scheme converges only for $\frac{\Delta t}{\Delta x} \leq 1$ with greater accuracy for smaller values of Δx & Δt .

Figure 3.1 Numerical and exact solution at $t = 5, 20, 35$ & 50 ($q = 0$).



For $q = 0$, Figure 3.1 compares the exact solution (3) with the numerical solution for $\frac{\Delta t}{\Delta x} = \frac{1}{2}$ and $\frac{\Delta t}{\Delta x} = 1$. We see that both numerical solutions closely match (3) however the numerical solution with $\frac{\Delta t}{\Delta x} = \frac{1}{2}$ begins to exhibit extraneous oscillatory behaviour as t increases, which was found to be the case whenever $\frac{\Delta t}{\Delta x} < 1$. The numerical solutions with $\frac{\Delta t}{\Delta x} = 1$ were found to be exceptionally accurate whilst the solutions with $\frac{\Delta t}{\Delta x} < 1$ were less so. Figure 3.2 illustrates the error of numerical solution at $t = 50$ for various values of $\frac{\Delta t}{\Delta x}$ and Δx , particularly in the region around $x = 50$, where the most significant errors occurred. Indeed, the errors take a very similar form at other values of t , but are of greater size at larger values of t . We see that if we reduce only Δx the errors get smaller, whilst if it is only Δt that is reduced the errors remain roughly the same.

Figure 3.2 Error of numerical solutions at $t = 50$ ($q = 0$).



For $q = 1$, Table 3.1 displays the error of the numerical solution at $t = 50$ for selected values of x , Δx & $\frac{\Delta t}{\Delta x}$, using (7) to calculate the exact solution. As with $q = 0$, the error always gets smaller when Δx is reduced. For x close to zero, reducing Δt can also reduce the error, albeit only up to a point. However, for larger x , as with for $q = 0$, it is $\frac{\Delta t}{\Delta x} = 1$ that gives the most accurate results. Indeed, the choice of steplength $\Delta x = \Delta t = 0.005$ was found to offer the greatest accuracy over the full range of x values whilst balancing computational cost, and subsequently this choice of steplength is used in the remainder of the project.

Table 3.1 *Error of numerical solution at $t = 50$ ($q = 1$).*

		$\Delta t = \Delta x$	$\Delta t = \frac{1}{2} \Delta x$	$\Delta t = \frac{1}{4} \Delta x$
$x = 1$	$\Delta x = 0.05$	-0.000131	-6.903×10^{-6}	2.227×10^{-5}
	$\Delta x = 0.01$	-5.198×10^{-6}	-3.233×10^{-7}	8.853×10^{-7}
	$\Delta x = 0.005$	-1.302×10^{-6}	-7.767×10^{-8}	2.230×10^{-7}
$x = 20$	$\Delta x = 0.05$	-7.000×10^{-5}	6.983×10^{-5}	0.000129
	$\Delta x = 0.01$	-2.812×10^{-6}	3.201×10^{-6}	5.171×10^{-6}
	$\Delta x = 0.005$	-7.017×10^{-7}	8.116×10^{-7}	1.284×10^{-6}
$x = 50$	$\Delta x = 0.05$	0.00134	-0.0133	-0.0178
	$\Delta x = 0.01$	6.178×10^{-5}	0.000516	0.000285
	$\Delta x = 0.005$	1.548×10^{-5}	0.000124	0.000152

Now finding $\tilde{f}(k)$:

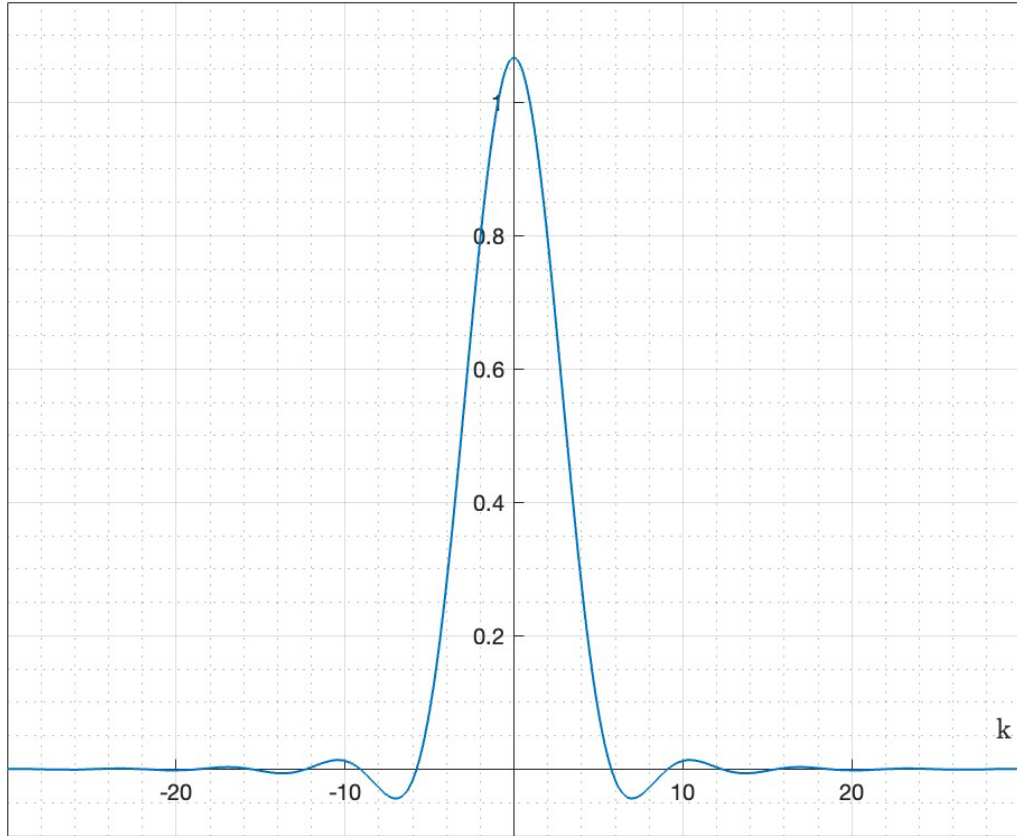
$$\begin{aligned}\tilde{f}(k) &= \int_{-\infty}^{\infty} e^{-ikx} f(x) dx \\ &= \int_{-1}^1 e^{-ikx} (1 - x^2)^2 dx .\end{aligned}$$

We proceed by repeated integration by parts:

$$= \left[\frac{i}{k} e^{-ikx} (1 - x^2)^2 \right]_{-1}^1 + \frac{4i}{k} \int_{-1}^1 e^{-ikx} x(1 - x^2) dx$$

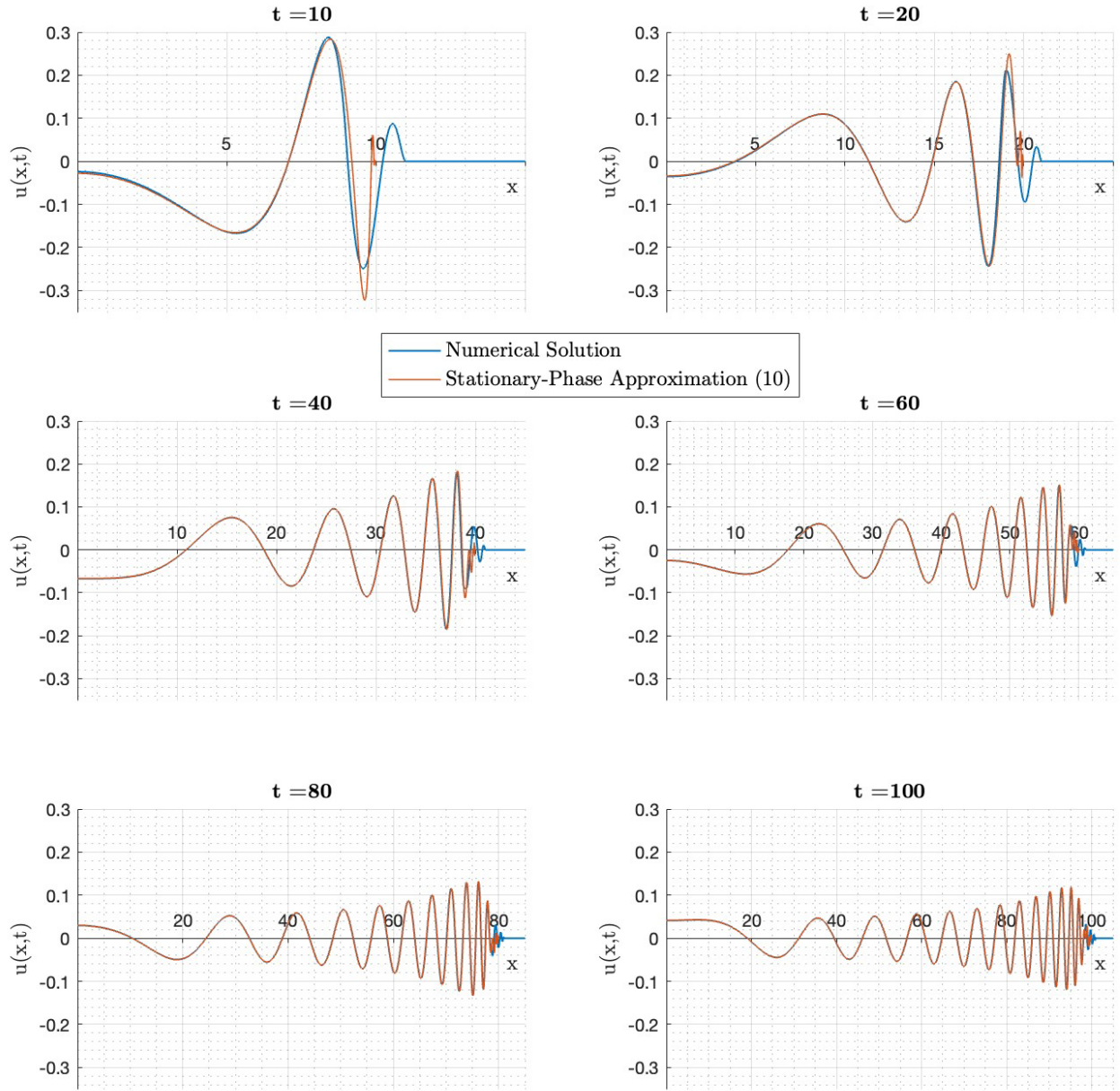
$$\begin{aligned}
&= \frac{4i}{k} \left[\frac{i}{k} e^{-ikx} x(1-x^2) \right]_{-1}^1 + \frac{4}{k^2} \int_{-1}^1 e^{-ikx} (1-3x^2) dx \\
&= \frac{4}{k^2} \left[\frac{i}{k} e^{-ikx} (1-3x^2) \right]_{-1}^1 + \frac{24i}{k^3} \int_{-1}^1 e^{-ikx} x dx \\
&= \frac{4i}{k^3} (2e^{ik} - 2e^{-ik}) + \frac{24i}{k^3} \left[\frac{i}{k} e^{-ikx} x \right]_{-1}^1 + \frac{24}{k^4} \int_{-1}^1 e^{-ikx} dx \\
&= -\frac{16}{k^3} \sin(k) - \frac{24}{k^4} (e^{-ik} + e^{ik}) + \frac{24i}{k^5} (e^{-ik} - e^{ik}) \\
&= -\frac{16}{k^3} \sin(k) - \frac{48}{k^4} \cos(k) + \frac{48}{k^5} \sin(k) \\
&= \frac{16}{k^3} \left(-\sin(k) - \frac{3 \cos(k)}{k} + \frac{3 \sin(k)}{k^2} \right)
\end{aligned}$$

Figure 3.3 Plot of $\tilde{f}(k)$



In Figure 3.4 we can see how, for $q = 1$, the numerical solution and stationary phase approximation closely match, with the only significant discrepancies occurring at x close to t . This is to be expected however, since (10) is valid only for $|x| < t$. Naturally, since (10) is an asymptotic approximation as $t \rightarrow \infty$, the agreement between the numerical solution and stationary phase approximation improves as t increases.

Figure 3.4 Numerical solution with $\Delta x = \Delta t = 0.005$ and stationary-phase approximation overlaid for various t ($q = 1$).



For both $q = 0$ and $q = 1$, the solution is in the form of a wave-packet moving away from the origin in both positive and negative x directions (although we have only been plotting the positive x direction). For $q = 0$, this wave-packet consists of precisely one (half) wave whereas for $q = 1$ the wave-packet is much broader, consisting of many smaller waves, with a long tail that extends to the origin. For $q = 0$, the wave-packet maintains its amplitude however for $q = 1$, the wave-packet's amplitude decreases with t and it becomes more spread out.

For a very small value of q we would expect a wave-packet that closely resembles what we see for $q = 0$, i.e. composed of small number of waves and peaked around $x = t$ with a thin (nearly zero) tail. We expect the solution to change continuously

as q decreases from $q = 1$ to $q = 0$ since q does not multiply any derivatives in (1) and hence the differential equation is not fundamentally changed if $q = 0$.

2 Signalling Problem

Question 4

A program solving (1) with the updated initial and boundary conditions is listed on page 21. In the same way as question 1, by rescaling $x \rightarrow x/c_0$ we can without loss of generality set $c_0 = 1$.

For $q = 0$ we have the problem:

$$\begin{aligned} u_{tt} - u_{xx} &= 0 & x, t > 0 \\ u(0, t) &= \sin(\omega_0 t) & t > 0 \\ u(x, 0) &= 0 & x > 0 \\ u_t(x, 0) &= 0 & x > 0 \end{aligned} \tag{4.1}$$

We first note that the two initial conditions are satisfied by setting $u(x, t) \equiv 0$ for $t < x$. Then for $x < t$, if we demand that $u(x, x) = 0$ for continuity at $t = x$, we find that the equation of motion and boundary condition at $x = 0$ are satisfied by $u(x, t) = -\sin(\omega_0(x - t))$. Putting this together, we have the solution to (4.1):

$$u(x, t) = \begin{cases} -\sin(\omega_0(x - t)) & 0 < x \leq t \\ 0 & t \leq x \end{cases}.$$

Indeed, as illustrated in Figure 4.1, the numerical solution for $q = 0$ agrees with this analytic solution.

For each of Figures 4.2 – 4.4, the numerical solution was calculated up to $x = 155$ with $\Delta x = \Delta t = 0.005$. In each case the solution is zero for $t > x$ and hence we can safely (maintaining numerical accuracy) specify $u_N^j = 0$ if $N\Delta x$ is larger than the value of t . As mentioned earlier, the finite-difference scheme with these steplengths gives very accurate solutions (considering our purposes) whilst balancing computational cost. In each case, as we have seen before, the solution consists of multiple smaller waves lying within a larger envelope which moves along the x -axis away from the origin.

Setting $q = 1$ results in a non-uniform envelope that decreases in amplitude as x approaches t . For $\omega_0 = 0.9$, the amplitude of the disturbance at $x = 0$ is not maintained in the envelope, which instead gets thinner as t increases. For $\omega_0 = 1.1$ & 1.5 the amplitude of the disturbance at $x = 0$ is maintained in the envelope up to a certain x value (about $x \approx \frac{2}{3}t$) before the amplitude of the envelope decays. This creates a sort of ‘nose’ in the envelope with stretches out as t increases.

Figure 4.1 Numerical solution with $\Delta x = \Delta t = 0.005$ and the analytic solution for various t ($q = 0$)

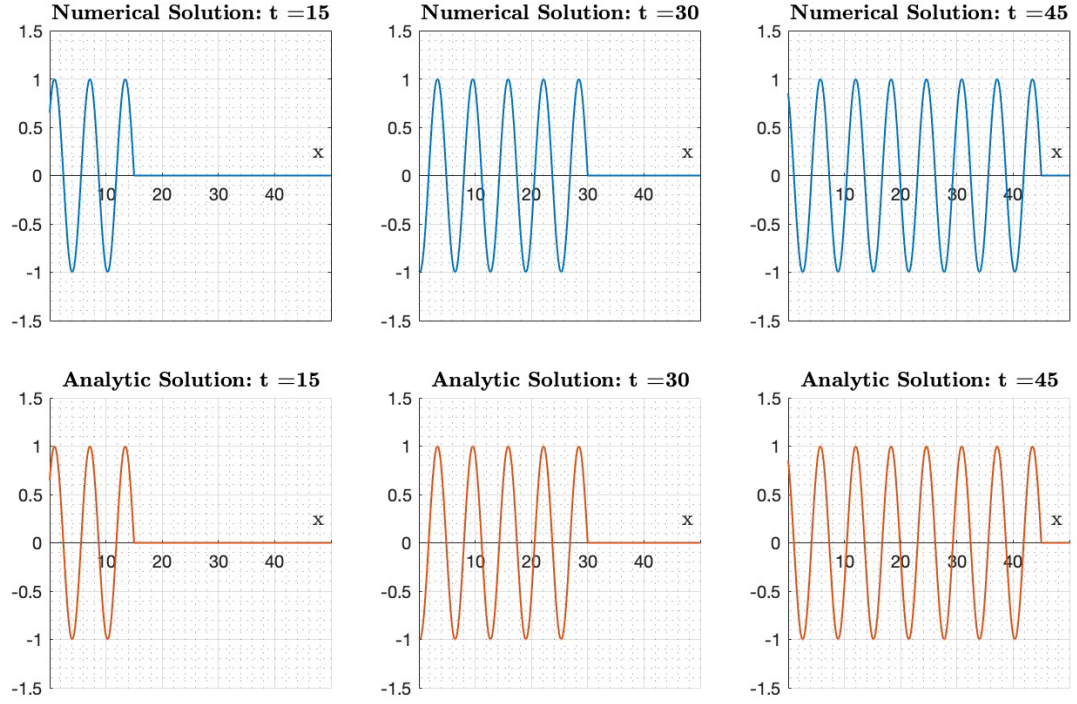
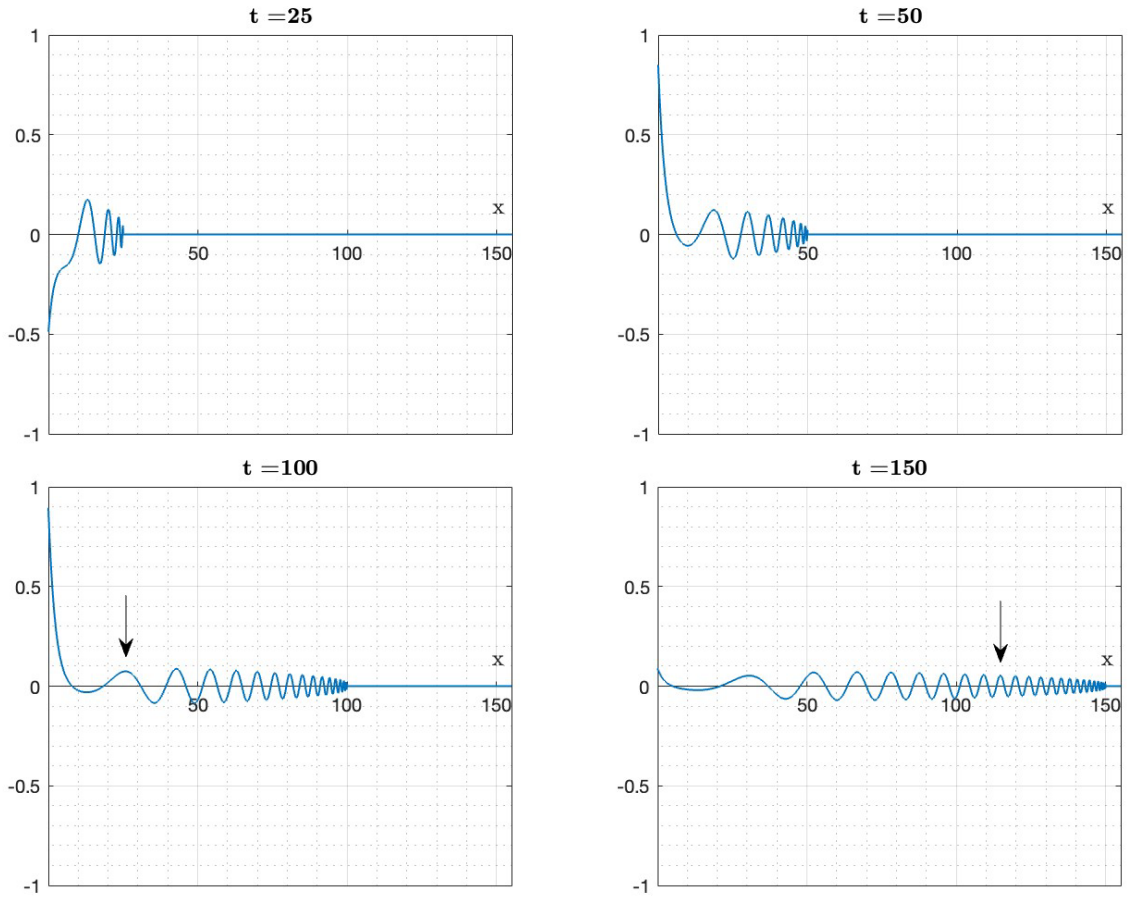


Figure 4.2 Numerical solution for $q = 1$ & $\omega_0 = 0.9$ with $\Delta x = \Delta t = 0.005$



For $q = 0$, we have $\frac{\Omega(k)}{k} = \Omega'(k) = 1$ for all k and hence the phase and group velocities are the same. This is why the envelope has a constant shape and does not spread out as it moves along the x -axis.

For $q = 1$, $\Omega(k)$ is a non-linear function of k and hence the phase and group velocities are not necessarily the same and will depend on the wavenumber k . This is why, in contrast to $q = 0$, the shape of the envelope changes at t increases. Indicated with an arrow in each of Figures 4.2 – 4.4 is the position of a particular wave peak at $t = 100$ and the same peak at $t = 150$. In each case, the wave peak has moved forward through the envelope towards the front, decreasing in amplitude as it does so. This indicates that, within the envelope, the phase velocity is greater than the group velocity which we know must be the case from looking at Figure 1.1 for $k > 0$. Hence the waves increase in density near the front of the envelope and have increasingly shorter wavelengths. Such shorter wavelengths correspond to a larger value of k and as illustrated in Figure 1.1 this results in the phase velocity and group velocity tending to 1 at the tip of the envelope.

Figure 4.3 Numerical solution for $q = 1$ & $\omega_0 = 1.1$ with $\Delta x = \Delta t = 0.005$

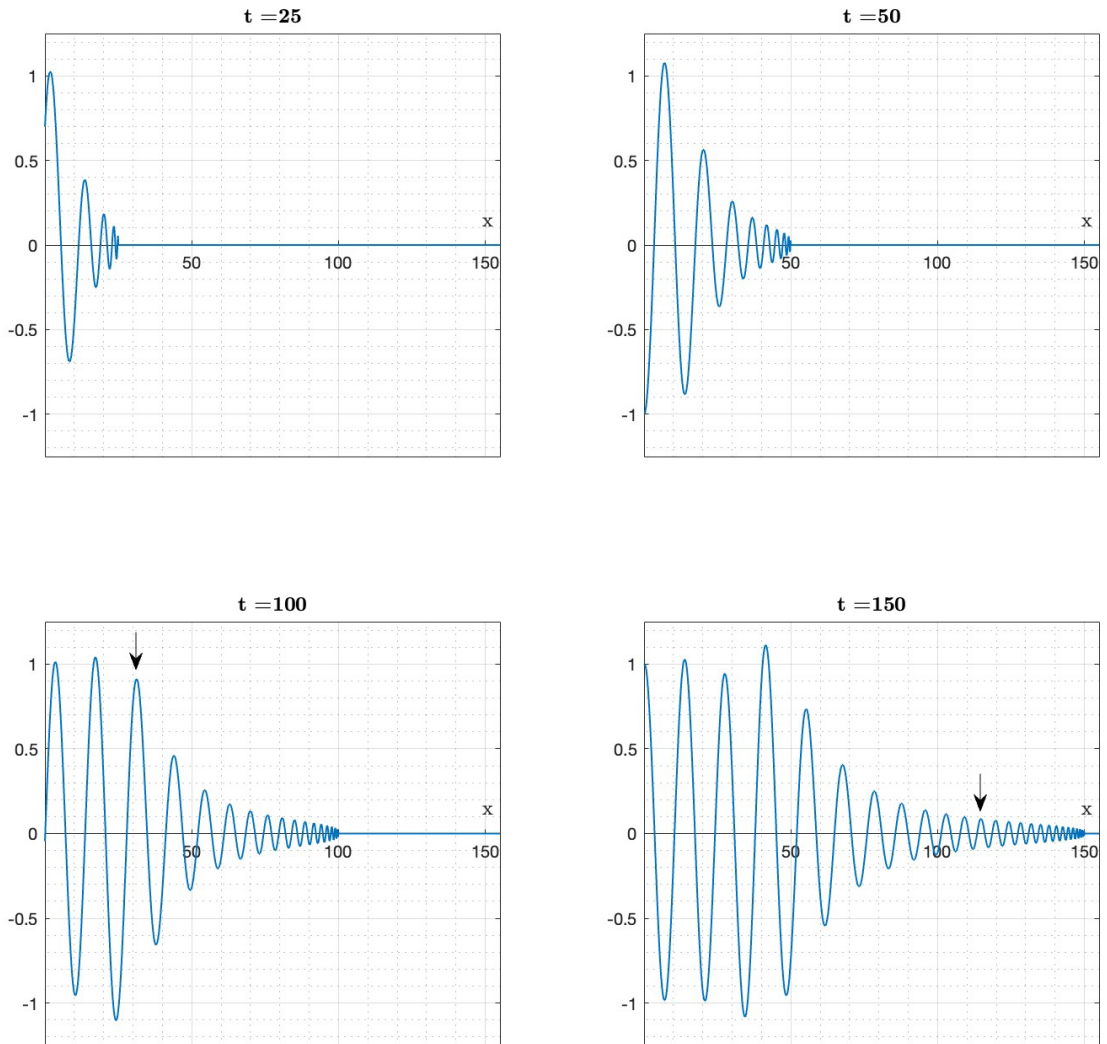
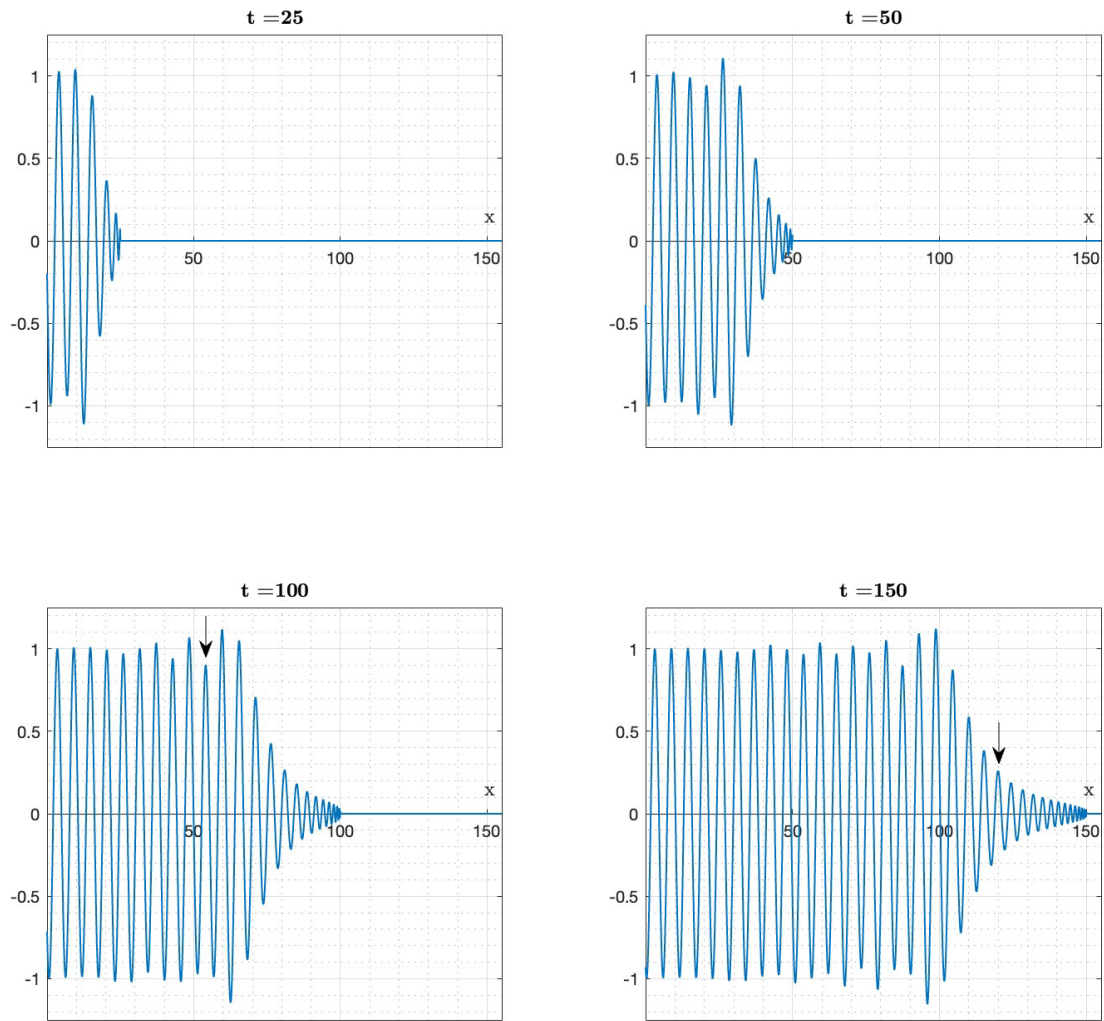


Figure 4.4 *Numerical solution for $q = 1$ & $\omega_0 = 1.5$ with $\Delta x = \Delta t = 0.005$*



Programs

All code written in MATLAB version R2024b with the use of the Symbolic Math Toolbox

Question 3

Program solving (1) – (2) numerically:

```
function U = Q3FiniteDiffSolver(xmax,tmax,deltax,deltat,q,f)
% Solves (1)–(2) by finite difference scheme
% INPUTS
% - xmax
% - tmax
% - deltax
% - deltat
% - q
% - f (input as an @(x) function)
% OUTPUTS
% - U, an array where U(i,j) is an approximation for
% u((i-2)*deltax,(j-2)*deltat)

Nx = xmax/deltax;
Nt = tmax/deltat;
ratio = (deltat^2)/(deltax^2);
coeff1 = ratio - (q^2)*(deltat^2)/2;
coeff2 = 2*(1 - ratio);
U = zeros(Nx+2,Nt+2);

% Note that the array U has indices shifted by 2 compared
% to the numerical solution u outlined in (12)
% U(i,j) = u(i-2, j-2)

% Boundary Conditions at x = N*deltax
U(Nx+2,:) = zeros(1, Nt+2);

% Initial Conditions (2): setting u(x,0) = f(x)
% this is the zeroth t-level
for i=1:Nx+1
    U(i,2) = f((i-2)*deltax);
end

% Solving first t-level using (12) and u(i,-1) = u(i,1)
% which corresponds to U(i,1) = U(i,3)
for i=2:Nx+1
    U(i,3) = (0.5)*coeff1*(U(i-1,2) + U(i+1,2)) + (0.5)*coeff2*U(i,2);
end
% Now implementing u(-1,j) = u(1,j)
U(1,3) = U(3,3);

% Solving for the remaining t-levels using (12)
for j=4:Nt+2
    for i=2:Nx+1
        U(i,j) = -U(i,j-2) + coeff1*(U(i-1,j-1) + U(i+1,j-1)) +
coeff2*U(i,j-1);
    end
    % Now implementing u(-1,j) = u(1,j)
    U(1,j) = U(3,j);
end
end
```


Program used to produce Figure 3.1

```
syms x
g = piecewise(abs(x) <= 1, (1-x^2)^2, abs(x) >= 1, 0);

f = @(x) subs(g,x);

u = @(x,t) (f(x-t)+f(x+t))/2;

xmax = 55;
tmax = 55;
teval = [5 20 35 50];
deltax1 = 0.05;
deltat1 = 0.025;
deltax2 = 0.05;
deltat2 = 0.05;

subplot(3,1,1)
hold on
for i = 1:numel(teval)-1
    plot(0:0.01:xmax, u(0:0.01:xmax, teval(i)), 'Color', '#D95319',
        'LineStyle', '--', 'LineWidth', 0.7)
end
plot(0:0.01:xmax, u(0:0.01:xmax, teval(end)), 'Color', '#D95319',
    'LineWidth', 1)
xlim([0 55])
ylim([0 0.6])
ax = gca;
ax.XAxisLocation = 'origin';
grid(ax, "on")
grid(ax, 'minor')
xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
ylabel('u(x,t)', 'FontName', 'CMU Serif', 'FontSize', 12)
title('Exact Solution (3)', 'FontName', 'CMU Serif', 'FontSize', 12)

subplot(3,1,2)
hold on
U = Q3FiniteDiffSolver(xmax,tmax,deltax1,deltat1,0,f);
for i = 1:numel(teval)-1
    plot(0:deltax1:xmax, U(2:end, teval(i)/deltat1 + 2), 'Color',
        '#0072BD', 'LineStyle', '--', 'LineWidth', 0.7)
end
plot(0:deltax1:xmax, U(2:end, teval(end)/deltat1 + 2), 'Color', '#0072BD',
    'LineWidth', 1)
ax = gca;
ax.XAxisLocation = 'origin';
xlim([0 55])
ylim([-0.1 0.6])
ax = gca;
ax.XAxisLocation = 'origin';
grid(ax, "on")
grid(ax, 'minor')
xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
ylabel('u(x,t)', 'FontName', 'CMU Serif', 'FontSize', 12)
title('Numerical Solution with  $\Delta t = 0.025$   $\Delta x = 0.05$ ', 'FontName', 'CMU
Serif', 'FontSize', 12)

subplot(3,1,3)
hold on
U = Q3FiniteDiffSolver(xmax,tmax,deltax2,deltat2,0,f);
```

```

for i = 1:numel(teval)-1
    plot(0:deltax2:xmax, U(2:end, teval(i)/deltat2 + 2), 'Color',
        '#0072BD', 'LineStyle', '--', 'LineWidth', 0.7)
end
plot(0:deltax2:xmax, U(2:end, teval(end)/deltat2 + 2), 'Color', '#0072BD',
    'LineWidth', 1)
xlim([0 55])
ylim([0 0.6])
ax = gca;
ax.XAxisLocation = 'origin';
grid(ax, "on")
grid(ax, 'minor')
xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
ylabel('u(x,t)', 'FontName', 'CMU Serif', 'FontSize', 12)
title('Numerical Solution with  $\Delta t = 0.05$   $\Delta x = 0.05$ ', 'FontName', 'CMU
Serif', 'FontSize', 12)

```

Program used to produce Figure 3.2

```

syms x
g = piecewise(abs(x) <= 1, (1-x^2)^2, abs(x) >= 1, 0);

f = @(x) subs(g,x);

u = @(x,t) (f(x-t)+f(x+t))/2;

xmax = 65;
tmax = 65;

deltax1 = [0.1 0.05 0.02];
deltat1 = [0.05 0.025 0.01];
deltax2 = [0.1 0.05 0.02];
deltat2 = [0.025 0.0125 0.005];
deltax3 = [0.1 0.05 0.02];
deltat3 = [0.0125 0.00625 0.0025];

teval = 50;

subplot(3,1,1)
hold on
for i=1:3
    U = Q3FiniteDiffSolver(xmax,tmax,deltax1(i),deltat1(i),0,f);
    plot(0:deltax1(i):xmax,abs(U(2:end,teval/deltat1(i) + 2)-
        transpose(u(0:deltax1(i):xmax,teval))), 'LineWidth', 1.2)
end
xlim([teval-5 teval+5])
ylim([0 0.15])
ax = gca;
ax.XAxisLocation = 'origin';
grid(ax, "on")
grid(ax, 'minor')
xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
ylabel('Error', 'FontName', 'CMU Serif', 'FontSize', 12)
legend('Δx = 0.1 Δt = 0.05', 'Δx = 0.05 Δt = 0.025', 'Δx = 0.02 Δt = 0.01',
    'FontName', 'CMU Serif', 'FontSize', 12)
title('Δt/Δx = 1/2', 'FontName', 'CMU Serif', 'FontSize', 14)

subplot(3,1,2)
hold on
for i=1:3

```

```

        U = Q3FiniteDiffSolver(xmax,tmax,deltax2(i),deltat2(i),0,f);
        plot(0:deltax2(i):xmax,abs(U(2:end,teval/deltat2(i) + 2)-
transpose(u(0:deltax2(i):xmax,teval))), 'LineWidth', 1.2)
    end
    xlim([teval-5 teval+5])
    ylim([0 0.15])
    ax = gca;
    ax.XAxisLocation = 'origin';
    grid(ax,"on")
    grid(ax,'minor')
    xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
    ylabel('Error', 'FontName', 'CMU Serif', 'FontSize', 12)
    legend('\Delta x = 0.1 \Delta t = 0.025', '\Delta x = 0.05 \Delta t = 0.0125', '\Delta x = 0.02 \Delta t = 0.005', 'FontName', 'CMU Serif', 'FontSize', 12)
    title('\Delta t/\Delta x = 1/4', 'FontName', 'CMU Serif', 'FontSize', 14)

subplot(3,1,3)
hold on
for i=1:3
    U = Q3FiniteDiffSolver(xmax,tmax,deltax3(i),deltat3(i),0,f);
    plot(0:deltax3(i):xmax,abs(U(2:end,teval/deltat3(i) + 2)-
transpose(u(0:deltax3(i):xmax,teval))), 'LineWidth', 1.2)
end
    xlim([teval-5 teval+5])
    ylim([0 0.15])
    ax = gca;
    ax.XAxisLocation = 'origin';
    grid(ax,"on")
    grid(ax,'minor')
    xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
    ylabel('Error', 'FontName', 'CMU Serif', 'FontSize', 12)
    legend('\Delta x = 0.1 \Delta t = 0.0125', '\Delta x = 0.05 \Delta t = 0.00625', '\Delta x = 0.02 \Delta t = 0.0025', 'FontName', 'CMU Serif', 'FontSize', 12)
    title('\Delta t/\Delta x = 1/8', 'FontName', 'CMU Serif', 'FontSize', 14)

```

Program used to produce Table 3.1

```

syms x
g = piecewise(abs(x) <= 1, (1-x^2)^2, abs(x) >= 1, 0);
f = @(x) subs(g,x);

xmax = 60;
tmax = 60;
deltax = [0.05;0.01;0.005];

xeval = ; % value of x changed manually (x = 1, 20 & 50)
teval = 50;

h = @(y) double(besselj(1,teval.*cos(y)).*subs(g, xeval + teval.*sin(y)));

u = (subs(g,xeval-teval)+subs(g,xeval+teval))/2 - (teval/2)*integral(h,-
pi/2,pi/2);

Error1 = zeros(3,1);
Error2 = zeros(3,1);
Error4 = zeros(3,1);
for i = 1:3
    U = Q3FiniteDiffSolver(xmax,tmax,deltax(i),deltax(i),1,f);
    Error1(i) = U(xeval/deltax(i) + 2, teval/deltax(i) + 2) - u;
end

```

```

for i = 1:3
    U = Q3FiniteDiffSolver(xmax,tmax,deltax(i),deltax(i)/2,1,f);
    Error2(i) = U(xeval/deltax(i) + 2, 2*teval/deltax(i) + 2) - u;
end
for i = 1:3
    U = Q3FiniteDiffSolver(xmax,tmax,deltax(i),deltax(i)/4,1,f);
    Error4(i) = U(xeval/deltax(i) + 2, 4*teval/deltax(i) + 2) - u;
end

table(deltax,Error1,Error2,Error4)

```

Program used to produce Figure 3.3

```

F = @(k) -(16*sin(k))/(k^3) - (48*cos(k))/(k^4) + (48*sin(k))/(k^5);

fplot(F, [-30 30], 'LineWidth',1, ShowPoles='off')
ax = gca;
ax.XAxisLocation = 'origin';
ax.YAxisLocation = 'origin';
xlim([-30 30])
ylim([-0.1 1.15])
xlabel("k", 'FontName', 'CMU Serif', 'FontSize', 13)

```

Program used to produce Figure 3.4

```

syms x k
g = piecewise(abs(x) <= 1, (1-x^2)^2, abs(x) >= 1, 0);
F = -(16*sin(k))/(k^3) - (48*cos(k))/(k^4) + (48*sin(k))/(k^5);
f = @(x) subs(g,x);

uasym = @(x,t) (t./(sqrt(2*pi)*((t.^2 - x.^2).^(3/4)))).*subs(F,
x./sqrt(t.^2 - x.^2)).*cos(sqrt(t.^2 - x.^2) + pi/4);

xmax = 110;
tmax = 110;
deltax = 0.005;
deltat = 0.005;
teval = [10 20 40 60 80 100];

U = Q3FiniteDiffSolver(xmax,tmax,deltax,deltat,1,f);

for i = 1:6
    subplot(3,2,i)
    hold on
    plot(0:deltax:xmax, U(2:end, teval(i)/deltat + 2),"LineWidth", 1)
    plot(deltax:deltax:teval(i)-deltax, uasym(deltax:deltax:teval(i)-
deltax,teval(i)),"LineWidth",0.8)
    ylim([-0.35 0.3])
    xlim([0 teval(i)+5])
    ax = gca;
    ax.XAxisLocation = 'origin';
    grid(ax,"on")
    grid(ax,'minor')
    xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
    ylabel('u(x,t)', 'FontName', 'CMU Serif', 'FontSize', 12)
    title(strcat('t = ',num2str(teval(i))), 'FontName', 'CMU Serif',
'FontSize', 12)
end
legend('Numerical Solution', 'Stationary-Phase Approximation (10)',
'FontName', 'CMU Serif', 'FontSize', 12)

```

Question 4

Program solving (1) with updated initial and boundary conditions:

```
function U = Q4FiniteDiffSolver(xmax,tmax,deltax,deltat,q,omega0)
% Solve (1) with new updated conditions by finite difference scheme
% INPUTS
% - xmax
% - tmax
% - deltax
% - deltat
% - q
% - omega0
% OUTPUTS
% - U, an array where U(i,j) is an approximation for
% u((i-2)*deltax,(j-2)*deltat)

Nx = xmax/deltax;
Nt = tmax/deltat;

ratio = (deltat^2)/(deltax^2);
coeff1 = ratio - (q^2)*(deltat^2)/2;
coeff2 = 2*(1 - ratio);
U = zeros(Nx+2,Nt+2);

% Note that the array U has indices shifted by 2 compared
% to the numerical solution u outlined in (12)
% U(i,j) = u(i-2, j-2)

% Boundary conditions at x = N*deltax
    U(Nx+2,:) = zeros(1, Nt+2);

% Boundary conditions at x = 0
    for j = 1:Nt+2
        U(2,j) = sin(omega0*(j-2)*deltat);
    end

% Initial conditions (2): setting u(x,0) = 0
% this is the zeroth t-level
    U(:,2) = zeros(Nx+2, 1);

% Solving first t-level using (12) and u(i,-1) = u(i,1)
% which corresponds to U(i,1) = U(i,3)
    for i=3:Nx+1
        U(i,3) = (0.5)*coeff1*(U(i-1,2) + U(i+1,2)) + (0.5)*coeff2*U(i,2);
    end

% Solving for the remaining t-levels using (12)
    for j=4:Nt+2
        for i=3:Nx+1
            U(i,j) = -U(i,j-2) + coeff1*(U(i-1,j-1) + U(i+1,j-1)) +
coeff2*U(i,j-1);
        end
    end
end
```

Program used to produce Figure 4.1

```
xmax = 50;
tmax = 50;
deltax = 0.005;
deltat = 0.005;
omega0 = 1;
U = Q4FiniteDiffSolver(xmax,tmax,deltax,deltat,0,omega0);
teval = [15 30 45];

for i = 1:3
    subplot(2,3,i)
    plot(0:deltax:xmax, U(2:end, teval(i)/deltat + 2),"LineWidth", 1)
    ylim([-1.5 1.5])
    xlim([0 50])
    xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
    ax = gca;
    ax.XAxisLocation = 'origin';
    grid(ax,"on")
    grid(ax,'minor')
    title(strcat('Numerical Solution: ',strcat(' t =',
num2str(teval(i))), 'FontName', 'CMU Serif', 'FontSize', 12)
    subplot(2,3,i+3)
    hold on
    plot(0:deltax:teval(i), -sin(omega0.*(-teval(i):deltax:0)), 'Color',
"#D95319", "LineWidth", 1)
    plot(teval(i):deltax:xmax, zeros(size(teval(i):deltax:xmax)), 'Color',
"#D95319", "LineWidth", 1)
    ylim([-1.5 1.5])
    xlim([0 50])
    xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
    ax = gca;
    ax.XAxisLocation = 'origin';
    grid(ax,"on")
    grid(ax,'minor')
    title(strcat('Analytic Solution: ',strcat(' t =', num2str(teval(i))),
'FontName', 'CMU Serif', 'FontSize', 12)
end
```

Program used to produce Figure 4.2

```
xmax = 155;
tmax = 155;
deltax = 0.005;
deltat = 0.005;
omega0 = 0.9;
teval = [25 50 100 150];
U = Q4FiniteDiffSolver(xmax,tmax,deltax,deltat,1,omega0);

for i = 1:4
    subplot(2,2,i)
    plot(0:deltax:xmax, U(2:end, teval(i)/deltat + 2),"LineWidth", 1)
    ylim([-1 1])
    xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
    ax = gca;
    ax.XAxisLocation = 'origin';
    grid(ax,"on")
    grid(ax,'minor')
    title(strcat('t =', num2str(teval(i))), 'FontName', 'CMU Serif',
'FontSize', 12)
end
```

Program used to produce Figure 4.3

```
xmax = 155;
tmax = 155;
deltax = 0.005;
deltat = 0.005;
omega0 = 1.1;
teval = [25 50 100 150];

U = Q4FiniteDiffSolver(xmax,tmax,deltax,deltat,1,omega0);

for i = 1:4
    subplot(2,2,i)
    plot(0:deltax:xmax, U(2:end, teval(i)/deltat + 2),"LineWidth", 1)
    ylim([-1.25 1.25])
    xlim([0 155])
    xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
    ax = gca;
    ax.XAxisLocation = 'origin';
    grid(ax,"on")
    grid(ax,'minor')
    title(strcat('t =', num2str(teval(i))), 'FontName', 'CMU Serif',
'FontSize', 12)
end
```

Program used to produce Figure 4.4

```
xmax = 155;
tmax = 155;
deltax = 0.005;
deltat = 0.005;
omega0 = 1.5;
teval = [25 50 100 150];

U = Q4FiniteDiffSolver(xmax,tmax,deltax,deltat,1,omega0);

for i = 1:4
    subplot(2,2,i)
    plot(0:deltax:xmax, U(2:end, teval(i)/deltat + 2),"LineWidth", 1)
    ylim([-1.25 1.25])
    xlim([0 155])
    xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
    ax = gca;
    ax.XAxisLocation = 'origin';
    grid(ax,"on")
    grid(ax,'minor')
    title(strcat('t =', num2str(teval(i))), 'FontName', 'CMU Serif',
'FontSize', 12)
end
```